

MITs — Project Status

Last updated: after Phase 6 (super admin). Phase 5 (CLI) and Phase 7 (deployment/pitch) remain.

1. What MITs is, in one line

A framework that lets a college generate a standardized, professional, admin-editable website for any club or department in minutes, while keeping all of them on one shared, centrally managed database.

2. Architecture at a glance

Layer	Choice	Why
Frontend framework	Next.js 16 (App Router), TypeScript	Real SSR, file-based routing, largest ecosystem — matters most for "others can maintain this later"
Styling	Tailwind CSS	Fast, consistent, token-driven
Database	PostgreSQL, running locally (<code>~/pg_data</code> , port 5432, no sudo)	Removes internet dependency during dev/demo; swappable connection string later if hosted
ORM	Prisma	One shared schema, type-safe queries
Auth	bcrypt-hashed passwords + JWT in httpOnly cookies, checked by Next.js middleware	Session token unreadable by browser JavaScript — real security improvement over localStorage
Multi-tenancy	One shared database, every table scoped by <code>organizationId</code>	Not a separate database per club — this is the actual "centralized" architecture

3. What's built and confirmed

Phase 0-1 — Database foundation

- Local PostgreSQL cluster running without root privileges.
- Database `mits_robotics`, user `mits_user`.
- Shared Prisma schema with models: `Organization`, `User` (org admins), `SuperAdmin`, `Event`, `Achievement`, `ActivityLog`.
- Seeded data: Robotics Club (with events, achievements, activity logs, admin login), IEDC MITS (org + admin login), one SuperAdmin account.

Note on IEDC MITS: this organization currently exists only as a manually seeded database row — it was **not** produced by the CLI, because the CLI doesn't exist yet. It's useful as test data for Phase 6, but it does not prove the "generate a new site" claim. Only Phase 5, once built and run, proves that.

Phase 2 — Public client site (`robotics/client`, port 3000)

- Fully migrated from Vite to Next.js App Router.
- Server Components query Prisma directly — zero client-side fetches for public page data.
- Brand colors injected server-side into the HTML `<head>` before the browser receives the page (no flash of default colors).
- Confirmed built: hero load-in sequence, animated stat counters, interactive event cards, responsive navigation.
- **Specified but not explicitly confirmed** — worth checking these are actually in the code, since they were requested but not mentioned in your own summary: scroll progress bar, page transitions, pinned About-page timeline, horizontal-scroll achievements gallery, magnetic buttons, Ken Burns image treatment, footer reveal, `prefers-reduced-motion` handling. Recommend a manual pass through the client site checking for each of these specifically.

Phase 3-4 — Org admin panel (`robotics/admin`, port 3002)

- Rebuilt as Next.js App Router, `server/` folder fully removed.
- Auth via `mits_session` httpOnly cookie, verified by middleware.
- Full CRUD for Events, Achievements, and Organization branding (colors, description, contact info).
- Every mutation writes to `ActivityLog` automatically.
- **Worth confirming:** the three required UI states (loading/empty/error), delete-confirmation dialogs, and success/error toasts from the Phase 4 spec — check these are present, not just the underlying CRUD logic.

Phase 6 — Super admin app (`superadmin`, port 3001)

- Standalone Next.js app, not tied to any organization.
- Separate `mits_superadmin_session` cookie and its own `SuperAdmin` table — cannot be confused with an org admin's session.
- Organization directory (`/`) — every org, event counts, last activity date.
- Drill-in view (`/organizations/[slug]`) — one org's events, achievements, recent activity.
- Global activity feed (`/activity`) — latest actions across every organization, one screen.
- Strictly read-only — no mutation capability anywhere in this app.

4. Currently running services

App	Port	Auth cookie	Tenant scope
<code>robotics/client</code>	3000	none (public)	Robotics Club only
<code>superadmin</code>	3001	<code>mits_superadmin_session</code>	All organizations
<code>robotics/admin</code>	3002	<code>mits_session</code>	Robotics Club only

There is currently no running site for IEDC MITS — only its database row exists.

5. What's NOT built yet

Phase 5 — The `mits-web` CLI (not started)

- `cli/` and `templates/club/` don't exist yet.
- The `mits-web create club <name>` command that generates a new, independent, differently-branded site in under a minute — the central claim of the whole project — has not been exercised.
- Until this exists, IEDC MITS being in the database proves nothing about the framework itself; it's just a second row of test data.

Phase 7 — Deployment, polish, pitch prep (not started)

- Nothing is deployed anywhere reachable outside your own machine yet.
- No README, architecture diagram, or demo video exists yet.
- No rehearsed walkthrough.

6. Before the demo, regardless of phase order

- Confirm the "specified but not explicitly confirmed" animation and admin UX items above are actually present — the gap between what was asked for and what was reported built is the main real risk right now.
- Run the tenant-isolation test properly once Phase 5 exists: create an event for a CLI-generated org, confirm it does not appear on Robotics Club's site, and vice versa.
- Check for hydration warnings in the browser console on all three currently running apps — a known Next.js-specific risk this project hasn't explicitly ruled out yet.

7. Recommended order from here

1. **Phase 5** — build the CLI, generate at least two new orgs with it, verify isolation. This is what turns "three hand-built apps" into an actual, provable framework.
2. **Phase 7** — deploy, polish, write the README and case study, rehearse the walkthrough.

Do not skip to Phase 7 before Phase 5 — the pitch's central claim depends on it.